C Exam Conspect

Pointers, enums, structs, files, dynamic memory, qsort, Makefile, logging/debugging.
Practical exam focus: `gcc`, `valgrind`, `make`, and correct memory/file handling.

Contents

1 Build, tools, and project organization **1**

2 Headers, include guards, and code division **1**

3 Accurate input handling and buffers **2**

4 Text files, CSV, and data modelling **2**

5 Enums **3**

6 Structs and nested structs **3**

7 Pointers **4**

8 Double pointers, arrays, and pointer arithmetic **4**

9 Dynamic memory: malloc, calloc, free **5**

10 Strings, sizeof, strlen, and ownership **5**

11 realloc and efficient dynamic arrays **5**

12 Full dynamic file-reading program **6**

13 Makefile for the full program **8**

14 Function pointers and qsort **9**

15 Logging and debugging basics **9**

1 Build, tools, and project organization

Idea. A C project should be divided into small files with clear responsibility. Header files describe the interface; source files implement the functions.

Exam theory. A `.h` file normally contains macros, enums, structs, typedefs, and function prototypes. A `.c` file contains function definitions. Splitting code improves readability, reuse, and build time, because a Makefile can recompile only files affected by changes.

```
gcc -o program main.c students.c
./program
valgrind ./program
valgrind --leak-check=full ./program
```

Listing 1: Minimal compile, run, and Valgrind usage

Important. For the exam, keep the tooling simple: know how to compile with `gcc`, run memory checks with `valgrind`, and automate build commands with a `Makefile`. Tools like `gdb`, ASan, and linters exist, but they are only a small side note here.

2 Headers, include guards, and code division

Idea. A header is the public contract of a module. Other files include it to know what types and functions are available.

```
#ifndef STUDENTS_H
#define STUDENTS_H

typedef struct {
    int id;
    char *firstName;
    char *lastName;
    double grade;
```

```
} Student;

void FreeStudents(Student *arr, int count);
int AppendStudent(Student **arr, int *count, int *cap,
                  int id, const char *first,
                  const char *last, double grade);
int ReadStudents(const char *fileName, Student **outArr, int *outCount);

#endif
```

Listing 2: students.h

Exam theory. Include guards prevent the same declarations from being processed multiple times. `#include <stdio.h>` is used for standard/system headers. `#include "students.h"` is used for your own headers. Avoid full paths in includes.

Important. Do not put ordinary function definitions or normal global variable definitions into headers. If a function body is placed in a header and included in several `.c` files, the linker may see several definitions of the same function.

3 Accurate input handling and buffers

Idea. A buffer is a temporary memory area used to receive input before validating, printing, or copying it into permanent storage.

Exam theory. C does not automatically know how much data the user or file will provide. A fixed array like `char name[64]` has space for 63 visible characters plus the final `'\0'`. If input writes more than the object can hold, the program has a buffer overflow. Overflow can corrupt nearby variables, heap metadata, return addresses, or simply produce wrong output. The result is undefined behavior, so the visible symptom may change between runs or systems.

```
char name[64];
scanf("%63s", name);       // max 63 chars + '\0'
```

Listing 3: Width-limited string read

Important. For `char buf[N]`, use `%Ns` with width `N-1`. Width is not optional decoration; it is what prevents writing past the buffer.

When can overflow happen?

- Reading with `%s` without width: `scanf("%s", buf)`.
- Copying a longer string into a smaller array with `strcpy`.
- Forgetting space for the string terminator `'\0'`.
- Writing past dynamically allocated memory, e.g. allocated `n` elements but writing index `n`.

4 Text files, CSV, and data modelling

Idea. Text files store characters. Ordinary files often use spaces or newlines as separators. CSV files use commas. In both cases, `fscanf` returns how many fields were successfully read, and that return value must be checked.

```
// data.txt: 101 Anna Smith 91.5
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) return 1;

int id;
char first[64], last[64];
double grade;
while (fscanf(fp, "%d %63s %63s %lf",
              &id, first, last, &grade) == 4) {
    printf("%d %s %s %.1f\n", id, first, last, grade);
}
fclose(fp);
```

Listing 4: Ordinary whitespace-separated file reading

```
// students.csv: 101,Anna,Smith,91.5
while (fscanf(fp, "%d,%63[^,],%63[^,],%lf",
              &id, first, last, &grade) == 4) {
    printf("%d %s %s %.1f\n", id, first, last, grade);
}
```

Listing 5: Simple CSV with fscanf

Exam theory. In `fscanf`, whitespace in the format usually means "skip any amount of whitespace". A literal comma in the format means the next input character must be a comma. The pattern `%63[^,]` reads up to 63 characters until a comma is reached.

Relational data idea

Relational data contains several connected entities. A primary key uniquely identifies one row. A foreign key refers to another entity's primary key. Example: `Person(id)` and `Vehicle(person_id)`. UML/ERD diagrams show entities, attributes, keys, and relationships. In C, a row is often represented as a struct, and several rows as an array of structs.

5 Enums

Idea. An enum is a set of named integer constants. It is useful for states, categories, modes, and error codes.

```
enum DevStatus {
    STATUS_OFF,
    STATUS_IDLE,
    STATUS_READY,
    STATUS_DONE
};

enum ErrorState {
    ERR_NONE,
    ERR_WRONG_STATE,
    ERR_NO_WATER
};
```

Listing 6: State and error enums

Exam theory. By default, enum values start at 0 and increase by 1. Manual values are allowed, and duplicate values are technically allowed. Enums are clearer than unrelated `#define` constants because they group related names and make code easier to read. Type checking is limited, because C still treats enum values mostly like integers.

Important. Enumerator names must be unique in their scope. Use prefixes such as `PUMP_OFF`, `CU_IDLE`, `ERR_NONE`. Without prefixes, two enums can both try to define `IDLE`; this causes a redeclaration error.

```
enum ErrorState IsReady(enum DevStatus st, float water)
{
    if (st != STATUS_IDLE) return ERR_WRONG_STATE;
    if (water < 15.0f) return ERR_NO_WATER;
    return ERR_NONE;
}
```

Listing 7: Enum returned from a function

6 Structs and nested structs

Idea. A struct groups related fields into one logical object. Members can be primitive types, arrays, pointers, enums, or other structs.

```
typedef struct {
    int id;
    char firstName[32];
    char lastName[32];
    double grade;
    enum DevStatus status;
} FixedStudent;

FixedStudent a = {101, "Anna", "Smith", 91.5, STATUS_READY};
FixedStudent b = {.id = 102, .grade = 78.0};
```

Listing 8: Struct declaration and initialization

Exam theory. Struct size can be larger than the sum of member sizes because the compiler can add padding for alignment. Struct assignment copies all members. If a member is an array, the array content is copied. If a member is a pointer, only the address is copied; the pointed data is not duplicated.

```
FixedStudent s;
FixedStudent *p = &s;

s.grade = 90.0;       // object.member
p->grade = 95.0;      // pointer->member
(*p).grade = 95.0;    // same meaning, uglier
```

Listing 9: Dot and arrow operators

```
typedef struct { unsigned day, month, year; } Date;

typedef struct {
    char name[32];
    Date contractDate;
} Person;

Person p;
p.contractDate.year = 2026;
```

Listing 10: Nested struct

Danger. If a struct contains `char *name`, copying the struct copies only the pointer. Two structs may then point to the same string. Changing through one changes the shared string, and freeing both can cause double free.

7 Pointers

Idea. A pointer stores a memory address. The pointer and the object it points to are separate objects.

```
int num = 25;
int *p = &num;

printf("%p\n", (void *)p);  // address stored in p
printf("%d\n", *p);         // value at that address
*p = 55;                    // changes num
```

Listing 11: Basic pointer operations

Exam theory. `&x` means address of `x`. `*p` dereferences pointer `p`. In a declaration, `int *p` declares a pointer. In an expression, `*p` accesses the pointed-to object. `NULL` means the pointer points to nothing useful. Dereferencing `NULL` or a dangling pointer is undefined behavior.

```
void SetVal(int *val)
{
    *val = 55;
}

int main(void)
{
    int num;
    int *p = &num;
    scanf("%d", p);     // p already contains &num
    SetVal(&num);       // function can modify num
    return 0;
}
```

Listing 12: Pointer with scanf and function parameter

Important. Variables are passed by value: the function receives a copy. Passing a pointer means the copied value is an address, so the function can access and modify the original object. Arrays passed to functions decay to pointers to their first element.

8 Double pointers, arrays, and pointer arithmetic

Idea. A double pointer stores the address of a pointer. It is used when a function must change the caller's pointer itself.

```
int AllocateData(int **data, int n)
{
    *data = malloc(sizeof(int) * n);
    if (*data == NULL) return 0;
    return 1;
}

int *arr = NULL;
if (AllocateData(&arr, 10)) {
    arr[0] = 5;
    free(arr);
}
```

Listing 13: Allocation through double pointer

Exam theory. Dereferencing once gives the inner pointer; dereferencing twice gives the final object. Pointer arithmetic moves in units of the pointed type. If `p` is `int *`, then `p + 1` points to the next `int`, not the next byte. Array indexing `p[i]` means `*(p+i)`.

```
int *p = malloc(sizeof(int) * 3);
p[0] = 5;
*(p + 1) = 3;
p[2] = 7;
free(p);
```

Listing 14: Pointer arithmetic and indexing

Important. Knowing an address does not extend lifetime. Do not return the address of a local variable. The local variable disappears when the function returns, even if you memorized its address .

9 Dynamic memory: malloc, calloc, free

Idea. Dynamic memory is allocated at runtime on the heap. Use it when size is unknown, large, changes during runtime, or must outlive the current function.

Exam theory. Local variables and local arrays usually live on the stack and are valid only in their scope. Dynamic memory remains valid until `free` is called, provided the program still has a pointer to it. Allocation can fail and return `NULL`. `malloc` returns uninitialized memory. `calloc` returns zero-initialized memory.

```
int *a = malloc(sizeof(int) * n);
if (a == NULL) return 1;

int *b = calloc(n, sizeof(int));
if (b == NULL) {
    free(a);
    return 1;
}

free(a);
free(b);
a = NULL;
b = NULL;
```

Listing 15: malloc, calloc, and free

Typical allocation workflow

1. Include `<stdlib.h>`.
2. Declare a pointer of suitable type.
3. Compute bytes using `sizeof`, not magic numbers.
4. Allocate memory.
5. Check for `NULL`.
6. Use memory only inside allocated bounds.
7. Free memory exactly once when no longer needed.

Danger. Memory leakage can be caused by forgetting `free`, losing the only pointer to allocated memory, returning early without cleanup, forgetting to free pointer members inside structs, or overwriting the old pointer with failed `realloc`.

10 Strings, sizeof, strlen, and ownership

Idea. A C string is a character sequence ending with `'\0'`. A `char *` can point to a string, but it does not automatically mean ownership.

```
char buf[128];
scanf("%127s", buf);

char *name = malloc(strlen(buf) + 1);
if (name == NULL) return 1;
strcpy(name, buf);

free(name);
```

Listing 16: Allocate exact memory for a string

Exam theory. `sizeof(arr)` gives the size in bytes of the actual array only in the scope where the array was declared. In a function parameter like `char arr[]` or `char *arr`, `sizeof(arr)` gives pointer size. `strlen(s)` counts characters before `'\0'`, not allocated capacity.

```
char arr[10] = "Hello!";
sizeof(arr);     // 10 bytes
strlen(arr);     // 6 characters
sizeof(char *);  // pointer size, often 8 on 64-bit
```

Listing 17: sizeof versus strlen

Important. Ownership means responsibility for freeing. If your code allocates memory for a string, your code must eventually free it exactly once. If you merely point to a string stored elsewhere, do not free it unless ownership was explicitly given.

11 realloc and efficient dynamic arrays

Idea. `realloc` changes the size of a dynamically allocated block. It may keep the same address, move data to a new address, or fail.

```
Student *tmp = realloc(students, sizeof(Student) * newCap);
if (tmp == NULL) {
    // students is still valid here
    FreeStudents(students, count);
    return 1;
}
students = tmp;
capacity = newCap;
```

Listing 18: Correct realloc pattern

Exam theory. Never write `data = realloc(data, newSize)` directly. If allocation fails, `NULL` is returned and the old pointer is lost, producing a memory leak. Growing an array by one element each time causes many reallocations. A better algorithm keeps `count` and `capacity`; when full, capacity doubles: `2,4,8,16,...`. For `N` inserted elements, this gives about `log2(N)` reallocations instead of `N` reallocations.

```
if (count == capacity) {
    int newCap = (capacity == 0) ? 2 : capacity * 2;
    Item *tmp = realloc(items, sizeof(Item) * newCap);
    if (tmp == NULL) { /* cleanup or recover */ }
    items = tmp;
    capacity = newCap;
}
items[count++] = newItem;
```

Listing 19: Capacity-doubling insertion idea

Important. Newly added memory after growing is uninitialized. Shrinking can discard data at the end. `realloc(NULL, size)` behaves like `malloc(size)`.

12 Full dynamic file-reading program

Idea. This project reads an ordinary whitespace-separated file using `fscanf`. The array of structs grows with capacity doubling. Each struct owns two dynamically allocated strings. Functions are split by responsibility so each part is easier to read and revise.

Input file

```
101 Anna Smith 91.5
102 Bob Miller 78.0
103 Clara Brown 88.2
```

Listing 20: data.txt

students.c: includes and string copy

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "students.h"

static char *CopyString(const char *src)
{
    char *dst = malloc(strlen(src) + 1);
    if (dst == NULL) return NULL;
    strcpy(dst, src);
    return dst;
}
```

Listing 21: students.c, part 1

students.c: cleanup function

```
void FreeStudents(Student *arr, int count)
{
    for (int i = 0; i < count; i++) {
        free(arr[i].firstName);
        free(arr[i].lastName);
    }
    free(arr);
}
```

Listing 22: students.c, part 2

students.c: append with capacity doubling

```
int AppendStudent(Student **arr, int *count, int *cap,
                  int id, const char *first,
                  const char *last, double grade)
{
    if (*count == *cap) {
        int newCap = (*cap == 0) ? 2 : (*cap * 2);
        Student *tmp = realloc(*arr, sizeof(Student) * newCap);
        if (tmp == NULL) return 0;
        *arr = tmp;
        *cap = newCap;
    }

    Student s;
    s.id = id;
    s.grade = grade;
    s.firstName = CopyString(first);
    s.lastName = CopyString(last);

    if (s.firstName == NULL || s.lastName == NULL) {
        free(s.firstName);
        free(s.lastName);
        return 0;
    }

    (*arr)[*count] = s;
    (*count)++;
    return 1;
}
```

Listing 23: students.c, part 3

students.c: reading from file

```
int ReadStudents(const char *fileName, Student **outArr, int *outCount)
{
    FILE *fp = fopen(fileName, "r");
    if (fp == NULL) return 0;

    Student *students = NULL;
    int count = 0, capacity = 0;
    int id;
    char firstBuf[128], lastBuf[128];
    double grade;

    while (fscanf(fp, "%d %127s %127s %lf",
                  &id, firstBuf, lastBuf, &grade) == 4) {
        if (!AppendStudent(&students, &count, &capacity,
                           id, firstBuf, lastBuf, grade)) {
            fclose(fp);
            FreeStudents(students, count);
            return 0;
        }
    }

    fclose(fp);
    *outArr = students;
    *outCount = count;
    return 1;
}
```

Listing 24: students.c, part 4

main.c

```
#include <stdio.h>
#include "students.h"

int main(void)
{
    Student *students = NULL;
    int count = 0;

    if (!ReadStudents("data.txt", &students, &count)) {
        fprintf(stderr, "Could not read data.txt\n");
        return 1;
    }

    for (int i = 0; i < count; i++) {
        printf("%d %s %s %.1f\n", students[i].id,
               students[i].firstName,
               students[i].lastName,
               students[i].grade);
    }

    FreeStudents(students, count);
    return 0;
}
```

Listing 25: main.c

Important. The buffers `firstBuf` and `lastBuf` are temporary. Every successful read overwrites them. Therefore each name is copied into separate heap memory before appending the struct. Cleanup must free `firstName`, `lastName`, and finally the struct array.

13 Makefile for the full program

Idea. The Makefile builds the program from multiple source files, runs it, checks it with Valgrind, and removes generated files.

```
CC = gcc
BIN = students
OBJ = main.o students.o

all: $(OBJ)
    $(CC) -o $(BIN) $(OBJ)

main.o: main.c students.h
    $(CC) -c main.c

students.o: students.c students.h
    $(CC) -c students.c

run: all
    ./$(BIN)

check: all
    valgrind --leak-check=full ./$(BIN)

.PHONY: all run check clean
clean:
    rm -f *.o $(BIN)
```

Listing 26: Makefile

Exam theory. A target has dependencies and commands. Commands must start with a real tab. The first target is the default when running `make`. `.PHONY` marks targets that do not create files named `run`, `check`, or `clean`.

14 Function pointers and qsort

Idea. A function has an address. A function pointer stores that address and can call the function indirectly.

```
int Add(int a, int b) { return a + b; }
int Sub(int a, int b) { return a - b; }

int Calc(int a, int b, int (*op)(int, int))
{
    return op(a, b);
}
```

Listing 27: Function pointer as parameter

Exam theory. Function pointers are used for callbacks and generic algorithms. `qsort` is a generic sorting function. It knows how to sort, but it does not know your data type, so you provide a comparison function.

```
#include <stdlib.h>

int CompareByGrade(const void *a, const void *b)
{
    const Student *s1 = a;
    const Student *s2 = b;
    if (s1->grade > s2->grade) return 1;
    if (s1->grade < s2->grade) return -1;
    return 0;
}

qsort(students, count, sizeof(students[0]), CompareByGrade);
```

Listing 28: qsort on Student structs

Important. The compare function returns negative if the first element should go before the second, zero if equal for ordering, and positive if it should go after. Avoid `return x-y;` for integers because subtraction can overflow. Typical `qsort` time is about `O(n log n)`, much better than `O(n^2)` bubble sort for large arrays.

15 Logging and debugging basics

Idea. Logging records program behavior. Debugging finds why behavior differs from expectation.

```
FILE *log = fopen("app.log", "a");
if (log != NULL) {
    fprintf(log, "Program started\n");
    fclose(log);
}
```

Listing 29: Simple file logging

Exam theory. Logs are used for traceability, debugging, and analytics. They are often timestamped and may have levels such as error, warning, and info. In small exam programs, simple `fprintf` messages are enough. Valgrind is useful for memory bugs: invalid read/write, uninitialized values, leaks, double free, and use-after-free.

Important. Always check `fopen`. Always close opened files. Run Valgrind after code using `malloc`, `calloc`, `realloc`, pointer members inside structs, and error paths. These are the most common places where memory bugs appear.